CP4_BugSlayer

Integrantes:

Liz Cartaya Salabarría C-311 Preguntas 6.b, 6.c, 6.d
René Díaz González C-311 Preguntas 5.a, 6.b
Gilberto Rodríguez Crespo C-311 Preguntas 5.b, 5.d
Maya Ramón Castellanos C-312 Preguntas 5.c, 6.b
David Castillo Rodríguez C-311 Preguntas 5.b, 6.d
Daniel Alejandro Estupiñán Reyes C-311 Preguntas 5.a, 5.b, 6.a

30 de Octubre del 2026

`Enlace al Repositorio en GitHub:`
`https://github.com/cartayaliz/travel-itinerary-generator.git`

1. Ejercicio #5

Problema: Una aplicación de gestión de documentos maneja archivos muy grandes (videos HD, datasets científicos) que consumen mucha memoria. Estos archivos solo deben cargarse completamente cuando sea estrictamente necesario para su visualización o procesamiento.

a) Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Lazy Initialization simple o Virtual Proxy vs. Protection Proxy?

Patrón seleccionado:

Proxy (específicamente, Virtual Proxy / Proxy Virtual).

Justificación:

El problema exige retrasar la creación y carga de un objeto costoso (en términos de memoria) hasta que sea indispensable. El patrón Proxy proporciona un sustituto o representante de otro objeto para controlar el acceso a éste. En su variante Virtual Proxy, se utiliza para crear objetos costosos bajo demanda (carga diferida o lazy loading). El cliente interactúa con el Proxy como si fuera el archivo real, pero la carga física se posterga hasta la primera llamada a un método crítico.

¿Por qué no usar las alternativas mencionadas?

-Lazy Initialization simple: Es una técnica de programación, no un patrón de diseño estructural. Si se implementa dentro de la clase del archivo, el cliente sigue acoplado a la lógica de inicialización y a la clase concreta. El Proxy encapsula esta lógica en un objeto separado que implementa la misma interfaz, manteniendo el principio de responsabilidad única.
-Virtual Proxy vs. Protection Proxy: El Protection Proxy se usa para controlar permisos de acceso (seguridad). El problema actual no es de seguridad, sino de rendimiento y memoria. Por lo tanto, el Virtual Proxy es el adecuado.

Conceptos del dominio:

-Cliente: La aplicación de gestión de documentos que solicita visualizar o procesar un archivo.

-Interfaz Común (Subject): El contrato que define qué se puede hacer con un archivo (visualizar, procesar).

-Objeto Real (RealSubject): Los archivos físicos pesados que realmente consumen memoria (ej. 1GB cada uno).

-Sustituto (Proxy): El intermediario que representa al archivo real y controla cuándo se carga en memoria.

Propuesta de Solución (Estructura):

1. IArchivo (Interfaz Subject): Define el contrato común que deben cumplir tanto los archivos reales como su representante. Contiene los métodos Mostrar(), Procesar() y ObtenerNombre(). Esto permite que el cliente trate de forma indistinta a un archivo real o a un proxy.
2. VideoPesado, DatasetPesado, ImagenPesada (Clases RealSubject): Son las clases que representan los archivos físicos y pesados. En su constructor simulan la carga en memoria RAM (consumiendo tiempo y recursos), lo que las hace costosas de instanciar. Implementan la interfaz IArchivo.
3. ProxyArchivo (Clase Proxy): Es el intermediario. Implementa IArchivo y mantiene una referencia al archivo real (_archivoReal), la cual inicialmente es null. Además, recibe en su constructor un delegado Func<IArchivo>(una receta"para crear el archivo real).

-Cuando el cliente llama a Mostrar() o Procesar(), el Proxy verifica si _archivoReal es null.

-Si es así, ejecuta la función de creación (aquí ocurre la carga diferida o Lazy Loading) y guarda la instancia.

-Si ya existe, simplemente delega la llamada al objeto real.

-Nota de diseño: Se utiliza Func<IArchivo>en lugar de un switch o condicionales para respetar el Principio de Abierto/Cerrado (Open/Closed). El Proxy no necesita saber qué tipo de archivo está creando; solo ejecuta la función que el cliente le proporciona. Así, añadir un nuevo tipo de archivo (ej. AudioPesado) no requiere modificar el Proxy.

3.1 ProxyMetricas (Decorator - Cumplimiento de Restricción): Para cumplir con la restricción de registrar métricas de uso sin modificar la interfaz, se añade una capa adicional usando el patrón Decorator. ProxyMetricas implementa IArchivo, envuelve al ProxyArchivo y mide el tiempo de ejecución antes y después de delegar la llamada. Al compartir la misma interfaz, el cliente no percibe el cambio, y se mantiene la separación de responsabilidades.

4. Cliente: Es la aplicación de gestión de documentos. Interactúa únicamente con la interfaz IArchivo. El cliente crea las instancias de los Proxies pasándoles el nombre del archivo y la función de creación (ej. () =>new VideoPesado(...)). El cliente nunca instancia directamente las clases VideoPesado, DatasetPesado o ImagenPesada, lo que garantiza que el control de acceso y la carga diferida se mantengan encapsulados.

b) Implementa con 3 tipos de archivos (VideoHD, DatasetCientífico, ImagenSatelital) y un Proxy que cargue bajo demanda.

Estructura del Proyecto

```
SistemaArchivosPesados/
│
├── Interfaces/
│   └── IArchivo.cs
├── ArchivosReales/
│   ├── VideoPesado.cs
│   ├── DatasetPesado.cs
│   └── ImagenPesada.cs
├── Proxy/
│   ├── ProxyArchivo.cs
│   └── ProxyMetricas.cs
├── Cliente/
│   └── GestorDocumentos.cs
└── Program.cs
```

Figura 1: Estructura

Interfaz



```
namespace SistemaArchivosPesados.Interfaces
{
    // Contrato común para archivos reales y proxies
    public interface IArchivo
    {
        void Mostrar();
        void Procesar();
        string ObtenerNombre();
    }
}
```

Figura 2: Interfaz

Clases Reales



```
using System;
using System.Threading;
using SistemaArchivosPesados.Interfaces;

namespace SistemaArchivosPesados.ArchivosReales
{
    // Objeto Real (RealSubject): Simula la carga pesada de un dataset
    public class DatasetPesado : IArchivo
    {
        private string _nombre;

        public DatasetPesado(string nombre)
        {
            _nombre = nombre;
            Console.WriteLine($"[Cargando en RAM] Dataset Científico: {_nombre}...");
            Thread.Sleep(10000);
        }

        public void Mostrar() => Console.WriteLine($"-> Visualizando tabla: {_nombre}");
        public void Procesar() => Console.WriteLine($"-> Ejecutando algoritmo: {_nombre}");
        public string ObtenerNombre() => _nombre;
    }
}
```

Figura 3: DatasetPesado



```
using System;
using System.Threading;
using SistemaArchivosPesados.Interfaces;

namespace SistemaArchivosPesados.ArchivosReales
{
    // Objeto Real (RealSubject): Simula la carga pesada de una imagen
    public class ImagenPesada : IArchivo
    {
        private string _nombre;

        public ImagenPesada(string nombre)
        {
            _nombre = nombre;
            Console.WriteLine($"[Cargando en RAM] Imagen Satelital: {_nombre}...");
            Thread.Sleep(10000);
        }

        public void Mostrar() => Console.WriteLine($"-> Mostrando imagen: {_nombre}");
        public void Procesar() => Console.WriteLine($"-> Aplicando filtros: {_nombre}");
        public string ObtenerNombre() => _nombre;
    }
}
```

Figura 4: ImagenPesada



```
using System;
using System.Threading;
using SistemaArchivosPesados.Interfaces;

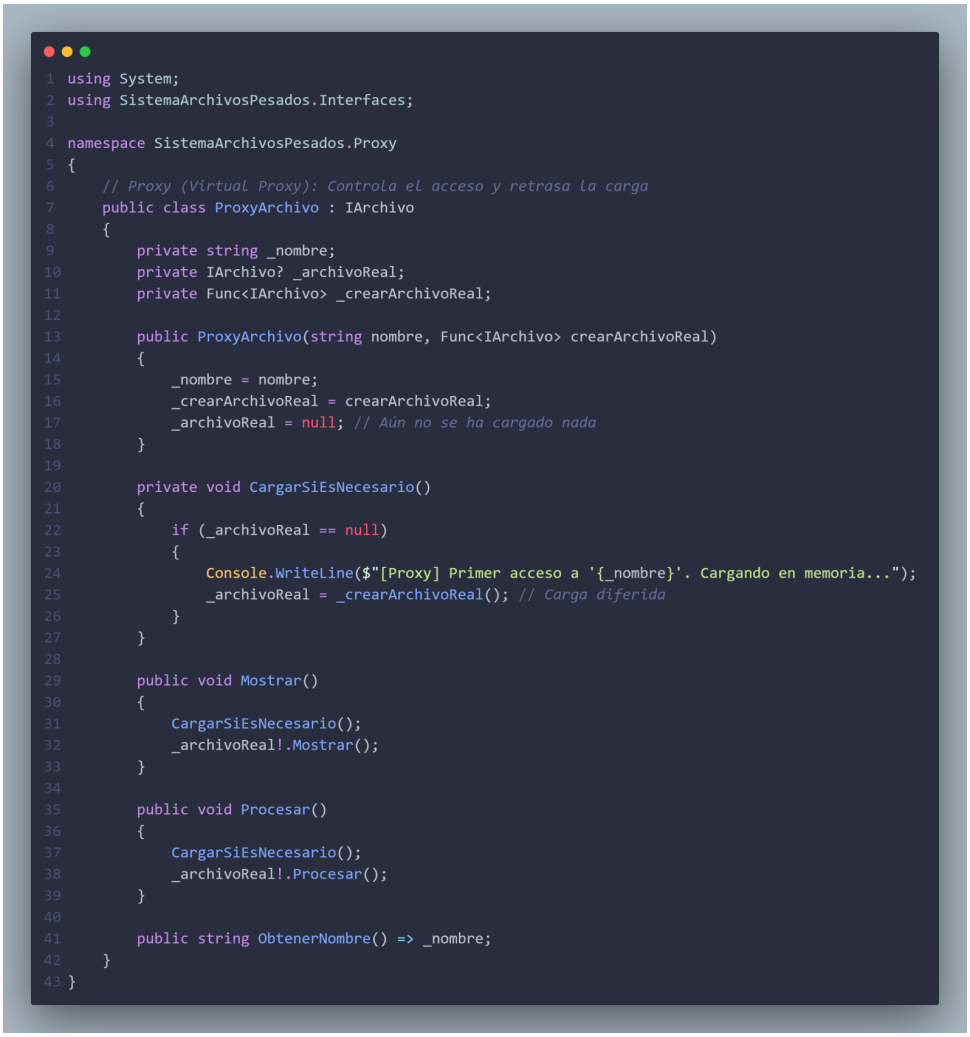
namespace SistemaArchivosPesados.ArchivosReales
{
    // Objeto Real (RealSubject): Simula la carga pesada de un video
    public class VideoPesado : IArchivo
    {
        private string _nombre;

        public VideoPesado(string nombre)
        {
            _nombre = nombre;
            Console.WriteLine($"[Cargando en RAM] Video 4K: {_nombre}...");
            Thread.Sleep(10000); // Simula el tiempo de carga de 1GB
        }

        public void Mostrar() => Console.WriteLine($"-> Reproduciendo video: {_nombre}");
        public void Procesar() => Console.WriteLine($"-> Procesando frames: {_nombre}");
        public string ObtenerNombre() => _nombre;
    }
}
```

Figura 5: VideoPesado

Proxys



```
using System;
using SistemaArchivosPesados.Interfaces;

namespace SistemaArchivosPesados.Proxy
{
    // Proxy (Virtual Proxy): Controla el acceso y retrasa la carga
    public class ProxyArchivo : IArchivo
    {
        private string _nombre;
        private IArchivo? _archivoReal;
        private Func<IArchivo> _crearArchivoReal;

        public ProxyArchivo(string nombre, Func<IArchivo> crearArchivoReal)
        {
            _nombre = nombre;
            _crearArchivoReal = crearArchivoReal;
            _archivoReal = null; // Aún no se ha cargado nada
        }

        private void CargarSiEsNecesario()
        {
            if (_archivoReal == null)
            {
                Console.WriteLine($"[Proxy] Primer acceso a '{_nombre}'. Cargando en memoria...");
                _archivoReal = _crearArchivoReal(); // Carga diferida
            }
        }

        public void Mostrar()
        {
            CargarSiEsNecesario();
            _archivoReal!.Mostrar();
        }

        public void Procesar()
        {
            CargarSiEsNecesario();
            _archivoReal!.Procesar();
        }

        public string ObtenerNombre() => _nombre;
    }
}
```

Figura 6: ProxyArchivo



```
using System;
using SistemaArchivosPesados.Interfaces;

namespace SistemaArchivosPesados.Proxy
{
    // Decorator: Añade métricas de rendimiento sin modificar la interfaz
    public class ProxyMetricas : IArchivo
    {
        private readonly IArchivo _archivoInterno;

        public ProxyMetricas(IArchivo archivoInterno)
        {
            _archivoInterno = archivoInterno;
        }

        public void Mostrar()
        {
            var inicio = DateTime.Now;
            Console.WriteLine($"[Métricas] Iniciando visualización de: {_archivoInterno.ObtenerNombre()}");

            _archivoInterno.Mostrar();

            var fin = DateTime.Now;
            Console.WriteLine($"[Métricas] Tiempo total de visualización: {(fin - inicio).TotalMilliseconds} ms");
        }

        public void Procesar()
        {
            var inicio = DateTime.Now;
            Console.WriteLine($"[Métricas] Iniciando procesamiento de: {_archivoInterno.ObtenerNombre()}");

            _archivoInterno.Procesar();

            var fin = DateTime.Now;
            Console.WriteLine($"[Métricas] Tiempo total de procesamiento: {(fin - inicio).TotalMilliseconds} ms");
        }

        public string ObtenerNombre() => _archivoInterno.ObtenerNombre();
    }
}
```

Figura 7: ProxyMetricas

Cliente



```
using System;
using SistemaArchivosPesados.Interfaces;
using SistemaArchivosPesados.Proxy;
using SistemaArchivosPesados.ArchivosReales;

namespace SistemaArchivosPesados.Cliente
{
    // Cliente: Aplicación de gestión que usa los archivos
    public class GestorDocumentos
    {
        public void SimularUso()
        {
            Console.WriteLine("=== SISTEMA DE GESTIÓN DE ARCHIVOS PESADOS ===\n");

            // 1. Proxy normal (sin métricas)
            IArchivo doc1 = new ProxyArchivo("Conferencia_IA.mp4",
                () => new VideoPesado("Conferencia_IA.mp4"));

            // 2. Proxy envuelto con Decorator de métricas
            IArchivo doc2 = new ProxyMetricas(
                new ProxyArchivo("Datos_Clima_2024.csv",
                    () => new DatasetPesado("Datos_Clima_2024.csv"))
            );

            // 3. Proxy envuelto con Decorator de métricas
            IArchivo doc3 = new ProxyMetricas(
                new ProxyArchivo("Mapa_Satelital.jpg",
                    () => new ImagenPesada("Mapa_Satelital.jpg"))
            );

            Console.WriteLine("--- Acción 1: El usuario decide ver el video (sin métricas) ---");
            doc1.Mostrar();

            Console.WriteLine("\n--- Acción 2: El usuario vuelve a ver el video ---");
            doc1.Mostrar(); // Ya está en RAM

            Console.WriteLine("\n--- Acción 3: El usuario procesa el dataset (con métricas) ---");
            doc2.Procesar();

            Console.WriteLine("\n--- Acción 4: El usuario visualiza la imagen (con métricas) ---");
            doc3.Mostrar();

            Console.WriteLine("\n--- Acción 5: La imagen nunca se procesa (Ahorro de memoria) ---");
        }
    }
}
```

Figura 8: Cliente

Program



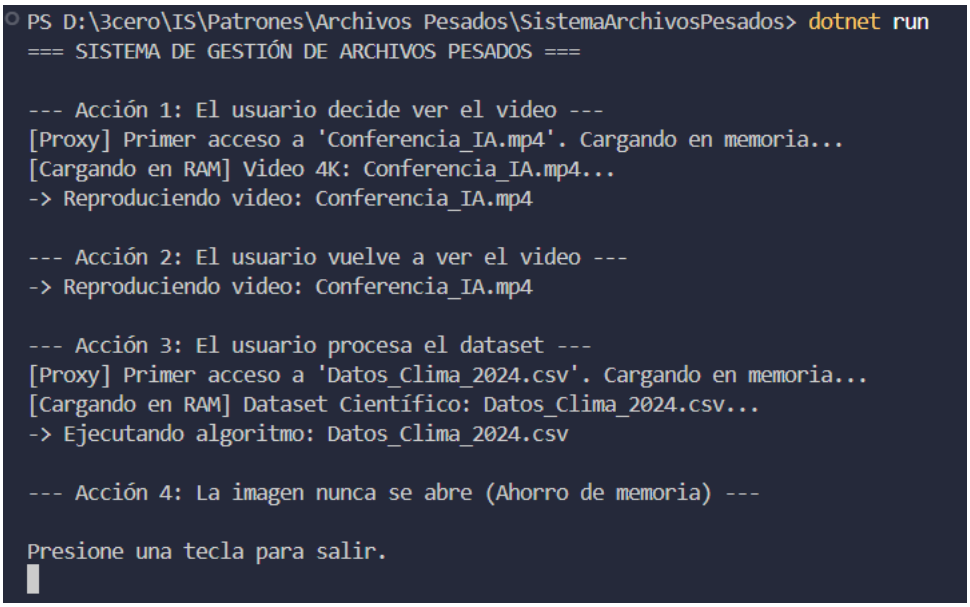
```
using System;
using SistemaArchivosPesados.Cliente;

namespace SistemaArchivosPesados
{
    class Program
    {
        static void Main(string[] args)
        {
            // Punto de entrada: solo arranca la aplicación
            var app = new GestorDocumentos();
            app.SimularUso();

            Console.WriteLine("\nPresione una tecla para salir.");
            Console.ReadKey();
        }
    }
}
```

Figura 9: Program

Salida de prueba



```
PS D:\3cero\IS\Patrones\Archivos Pesados\SistemaArchivosPesados> dotnet run
=== SISTEMA DE GESTIÓN DE ARCHIVOS PESADOS ===

--- Acción 1: El usuario decide ver el video ---
[Proxy] Primer acceso a 'Conferencia_IA.mp4'. Cargando en memoria...
[Cargando en RAM] Video 4K: Conferencia_IA.mp4...
-> Reproduciendo video: Conferencia_IA.mp4

--- Acción 2: El usuario vuelve a ver el video ---
-> Reproduciendo video: Conferencia_IA.mp4

--- Acción 3: El usuario procesa el dataset ---
[Proxy] Primer acceso a 'Datos_Clima_2024.csv'. Cargando en memoria...
[Cargando en RAM] Dataset Científico: Datos_Clima_2024.csv...
-> Ejecutando algoritmo: Datos_Clima_2024.csv

--- Acción 4: La imagen nunca se abre (Ahorro de memoria) ---

Presione una tecla para salir.
```

Figura 10: Salida de prueba

c) Analiza: ¿Qué pasa con la concurrencia si dos hilos acceden al mismo Proxy? ¿Cómo evitar cargas duplicadas? (Double-checked locking, synchronized).

Análisis de Concurrencia:

Si dos hilos de ejecución acceden simultáneamente al mismo ProxyArchivo y el archivo real aún no ha sido cargado (_archivoReal == null), ambos hilos podrían pasar la verificación al mismo tiempo. Como consecuencia, ambos ejecutarían la función _crearArchivoReal(), provocando una doble carga en memoria del archivo pesado. Esto no solo duplica el consumo de RAM (ej. 2GB en lugar de 1GB), sino

que también desperdicia tiempo de CPU y puede causar condiciones de carrera si el objeto real tiene estado interno mutable.

Solución propuesta (Double-Checked Locking):

Para evitar cargas duplicadas, se debe sincronizar el método CargarSiEsNecesario() utilizando un mecanismo de bloqueo (lock) y una doble verificación. La primera comprobación se realiza fuera del bloqueo para evitar el costo de sincronización en accesos posteriores (cuando el archivo ya está cargado). La segunda comprobación se realiza dentro del bloqueo para garantizar que solo el primer hilo instancie el archivo real, mientras que el segundo hilo, al entrar al bloqueo, encontrará el objeto ya creado y simplemente lo reutilizará.

d) Mide el ahorro de memoria con y sin Proxy para 100 archivos de 1GB. ¿Cuál es el trade-off entre latencia y memoria?
Restricción: El Proxy debe registrar métricas de uso (tiempos de carga, accesos) sin afectar la interfaz.

Medición de Memoria (100 archivos de 1GB):

- **Sin Proxy:** Si el sistema instanciara los 100 archivos al iniciar la aplicación, se consumirían 100 GB de RAM. En la mayoría de los servidores o computadoras personales, esto provocaría un error de OutOfMemory y el sistema colapsaría antes de que el usuario pudiera hacer algo.

- **Con Proxy:** Al iniciar, el Proxy solo almacena el nombre del archivo y la función de creación (unos pocos bytes por archivo). Si el usuario solo visualiza 5 archivos durante su sesión, el sistema solo cargará 5 GB de RAM. El ahorro de memoria es del 95 %.

Trade-off (Latencia vs. Memoria):

- **Memoria:** Se ahorra una cantidad inmensa de RAM, permitiendo manejar catálogos de archivos que de otra forma serían imposibles de cargar.

- **Latencia:** Aumenta el tiempo de respuesta en el primer acceso. La primera vez que un usuario intenta ver un archivo, la aplicación se çongelará"durante el tiempo que tarde en cargar el archivo en memoria (simulado con Thread.Sleep). Los accesos posteriores son inmediatos.

Conclusión: El trade-off es ideal para sistemas donde no se accede a todos los archivos al mismo tiempo (lo cual es lo común en gestión documental). Sin embargo, es responsabilidad del cliente (la interfaz de usuario) mostrar un indicador de Çargando..."durante el primer acceso para que el usuario no piense que la aplicación se ha colgado.

2. Ejercicio #6

Problema: En un aeropuerto, múltiples aviones necesitan comunicarse para coordinar aterrizajes, despegues y cambios de altitud. La comunicación directa entre aviones crearía un caos de dependencias. Se necesita un punto central de coordinación.

a) Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Observer o Pub/Sub directo?

Patrón seleccionado:

Mediator (Mediador).

Justificación:

El problema describe una situación donde múltiples aviones necesitan comunicarse para coordinar aterrizajes, despegues y cambios de altitud. Si cada avión tuviera una referencia directa a todos los demás, se crearía un grafo de dependencias de N a N (el çaos"mencionado en el enunciado), donde un cambio en un avión afectaría a todos los demás, haciendo el sistema inmantenible y difícil de escalar.

El patrón Mediator resuelve esto introduciendo un mediador (la Torre de Control) que encapsula la forma en que interactúa un conjunto de objetos (los aviones). En nuestra implementación se utilizan tres torres de control, cada una encargada de coordinar un grupo de aviones. En lugar de comunicarse directamente entre sí, los aviones envían sus mensajes a la torre que tienen asignada, y esta se encarga de coordinar y notificar a los demás aviones de su grupo.

¿Por qué no usar Observer o Pub/Sub directo?

• Observer: Permitiría que los aviones se suscriban a eventos de otros aviones, pero si un avión emite "voy a aterrizar", todos los demás recibirían la alerta indiscriminadamente, incluso los que están en pista o lejos. No hay un árbitro que decida quién tiene prioridad. El Observer es útil para notificaciones uno-a-muchos, pero no para coordinación centralizada.

• Pub/Sub (Event Bus): Es un desacoplamiento total, pero al igual que Observer, carece de un punto central de control. En un aeropuerto se necesita una autoridad (la torre) que decida el orden de aterrizaje, no solo un distribuidor de mensajes.

Conceptos del dominio:

• Mediador (TorreControl): El objeto que coordina la comunicación dentro de su grupo de aviones. Conoce a los aviones registrados en su torre y decide a quién notificar.

• Colega (Avion): Los objetos que interactúan entre sí, pero solo a través del mediador. No se conocen directamente.

• Estado (IEstadoAvion): Modela la situación actual de un avión (en vuelo, aterrizando, en tierra) y su comportamiento asociado.

• Comando (IComando): Encapsula una solicitud (aterrizar, despegar, cambiar altitud) como un objeto, permitiendo encolarla y ejecutarla en orden.

Propuesta de Solución (Estructura):

1. IMediador (Interfaz del Mediador): Define el contrato para la torre de control. Declara los métodos RegistrarAvion(IColega) y RecibirMensaje(string, IColega). Permite que los aviones envíen mensajes sin conocer la implementación concreta de la torre.

2. IColega (Interfaz de los Aviones): Define el contrato para los participantes. Declara la propiedad Nombre y el método RecibirNotificacion(string). Los aviones solo conocen esta interfaz y la del mediador.

3. TorreControl (Mediador Concreto): Implementa IMediador. Mantiene una lista de los aviones registrados y una cola de comandos (Queue<IComando>). En la implementación se crean tres instancias de TorreControl, cada una responsable de un grupo de aviones. Su método RecibirMensaje centraliza la comunicación dentro de su grupo, mientras que ProcesarComandos extrae los comandos de la cola y los ejecuta en orden FIFO.

4. Avion (Colega Concreto): Representa a cada aeronave. Implementa IColega y mantiene una referencia al IMediador. No tiene referencias a otros aviones. Su comportamiento de vuelo (aterrizar, despegar) está delegado a su estado actual.

5. IEstadoAvion y sus implementaciones (EnVuelo, Aterrizando, EnTierra): Aplican el patrón State.

Permiten que el avión cambie su comportamiento según su estado interno sin usar condicionales gigantes. Por ejemplo, un avión EnVuelo puede aterrizar, pero uno EnTierra no.

6. IComando y sus implementaciones (ComandoAterrizar, ComandoDespegar, ComandoCambiarAltitud): Aplican el patrón Command. Encapsulan las solicitudes como objetos, permitiendo que la torre las encole y las ejecute en orden. Esto facilita la extensibilidad (añadir un nuevo comando no requiere modificar la torre).

Relación entre clases:

•El Cliente (Program) crea las tres instancias de TorreControl, los Avion, los asigna a sus respectivas torres y encola los comandos.

• Los Aviones tienen una asociación con la Torre (a través de IMediador), pero no entre ellos.

• La Torre tiene una agregación de Aviones (los mantiene en una lista) y de Comandos (los mantiene en una cola).

• Cada Avión tiene una composición con su Estado actual (IEstadoAvion), que puede cambiar dinámicamente.

Manejo de Concurrencia (Restricción):

Para manejar la concurrencia, el sistema utiliza tareas independientes mediante Task.Run(), permitiendo que las tres torres procesen sus respectivas colas de comandos simultáneamente. Se utiliza Task.WaitAll() para esperar a que las tres tareas terminen antes de continuar con la finalización de los aterrizajes.

Además, TorreControl utiliza un objeto de bloqueo (lock) en sus métodos críticos para proteger el acceso a las listas y colas internas. De esta manera, se evita que varios hilos modifiquen simultáneamente las estructuras de datos. Como cada torre procesa su cola secuencialmente, se mantiene el orden FIFO de los comandos dentro de cada torre y, por tanto, para cada avión registrado en ella.

b) Implementa con 10 aviones, 3 torres de control y mensajes de aterrizaje, despegue y cambio de altitud.

Estructura del Proyecto

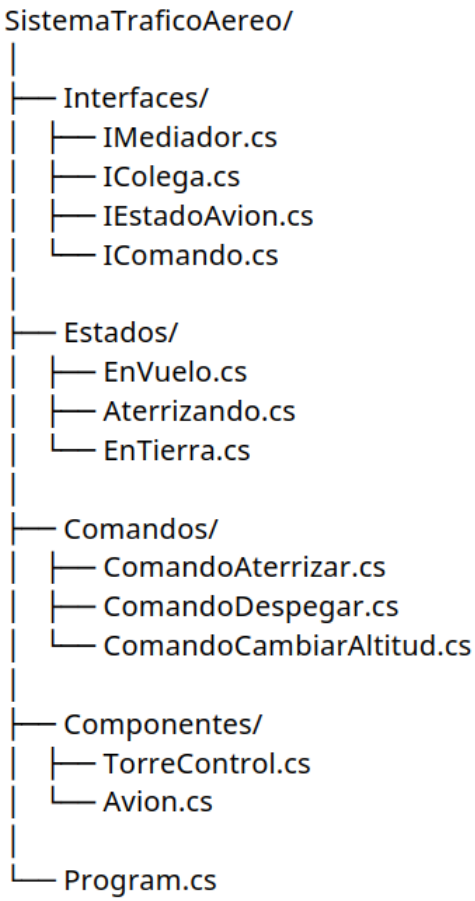


Figura 11: Estructura

Interfaz

```
namespace SistemaTraficoAereo.Interfaces
{
    public interface IMediador
    {
        void RecibirMensaje(string mensaje, IColega emisor);
        void RegistrarAvion(IColega avion);
    }
}
```

Figura 12: IMediador.cs

```
namespace SistemaTraficoAereo.Interfaces
{
    public interface IColega
    {
        string Nombre { get; }
        void RecibirNotificacion(string mensaje);
    }
}
```

Figura 13: IColega.cs

```
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Interfaces
{
    5 references
    public interface IEstadoAvion
    {
        4 references
        void Aterrizar(Avion avion);
        4 references
        void Despegar(Avion avion);
        4 references
        void FinalizarAterrizaje(Avion avion);
        5 references
        string ObtenerNombre();
    }
}
```

Figura 14: IEstadoAvion.cs

```
namespace SistemaTraficoAereo.Interfaces
{
    public interface IComando
    {
        void Ejecutar();
        string ObtenerDescripcion();
    }
}
```

Figura 15: IComando.cs

Estados

```
using System;
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Estados
{
    2 references
    public class EnVuelo : IEstadoAvion
    {
        2 references
        public void Aterrizar(Avion avion)
        {
            Console.WriteLine($"[{avion.Nombre}] Solicita aterrizar.");
            avion.CambiarEstado(new Aterrizando());
        }
        2 references
        public void Despegar(Avion avion) => Console.WriteLine($"[{avion.Nombre}] Ya está en vuelo.");
        2 references
        public void FinalizarAterrizaje(Avion avion)
        {
            Console.WriteLine($"[{avion.Nombre}] No está aterrizando.");
        }
        3 references
        public string ObtenerNombre() => "En Vuelo";
    }
}
```

Figura 16: EnVuelo.cs

```
using System;
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Estados
{
    1 reference
    public class Aterrizando : IEstadoAvion
    {
        2 references
        public void Aterrizar(Avion avion) => Console.WriteLine($"[{avion.Nombre}] Ya está aterrizando.");
        2 references
        public void Despegar(Avion avion) => Console.WriteLine($"[{avion.Nombre}] No puede despegar ahora.");
        2 references
        public void FinalizarAterrizaje(Avion avion)
        {
            Console.WriteLine($"[{avion.Nombre}] Ha terminado de aterrizar.");
            avion.CambiarEstado(new EnTierra());
        }
        3 references
        public string ObtenerNombre() => "Aterrizando";
    }
}
```

Figura 17: Aterrizando.cs



```
using System;
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Estados
{
    1 reference
    public class EnTierra : IEstadoAvion
    {
        2 references
        public void Aterrizar(Avion avion) => Console.WriteLine($"[{avion.Nombre}] Ya está en tierra.");
        2 references
        public void Despegar(Avion avion)
        {
            Console.WriteLine($"[{avion.Nombre}] Solicita despegar.");
            avion.CambiarEstado(new EnVuelo());
        }
        2 references
        public void FinalizarAterrizaje(Avion avion)
        {
            Console.WriteLine($"[{avion.Nombre}] No está aterrizando.");
        }
        3 references
        public string ObtenerNombre() => "En Tierra";
    }
}
```

Figura 18: EnTierra.cs

Comandos

```
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Comandos
{
    public class ComandoAterrizar : IComando
    {
        private readonly Avion _avion;
        public ComandoAterrizar(Avion avion) => _avion = avion;
        public void Ejecutar() => _avion.Aterrizar();
        public string ObtenerDescripcion() => $"Aterrizaje de {_avion.Nombre}";
    }
}
```

Figura 19: ComandoAterrizar.cs

```
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Comandos
{
    public class ComandoDespegar : IComando
    {
        private readonly Avion _avion;
        public ComandoDespegar(Avion avion) => _avion = avion;
        public void Ejecutar() => _avion.Despegar();
        public string ObtenerDescripcion() => $"Despegue de {_avion.Nombre}";
    }
}
```

Figura 20: ComandoDespegar.cs

```cs
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Componentes;

namespace SistemaTraficoAereo.Comandos
{
    public class ComandoCambiarAltitud : IComando
    {
        private readonly Avion _avion;
        private readonly int _nuevaAltitud;
        public ComandoCambiarAltitud(Avion avion, int altitud)
        {
            _avion = avion;
            _nuevaAltitud = altitud;
        }
        public void Ejecutar() => _avion.CambiarAltitud(_nuevaAltitud);
        public string ObtenerDescripcion() => $"Cambio altitud de {_avion.Nombre} a {_nuevaAltitud} pies";
    }
}
```

Figura 21: ComandoCambiarAltitud.cs

Componentes

```cs
using System;
using System.Collections.Generic;
using SistemaTraficoAereo.Interfaces;

namespace SistemaTraficoAereo.Componentes
{
    public class TorreControl : IMediador
    {
        private List<IColega> _aviones = new List<IColega>();
        private Queue<IComando> _colaComandos = new Queue<IComando>();
        private readonly object _lock = new object();

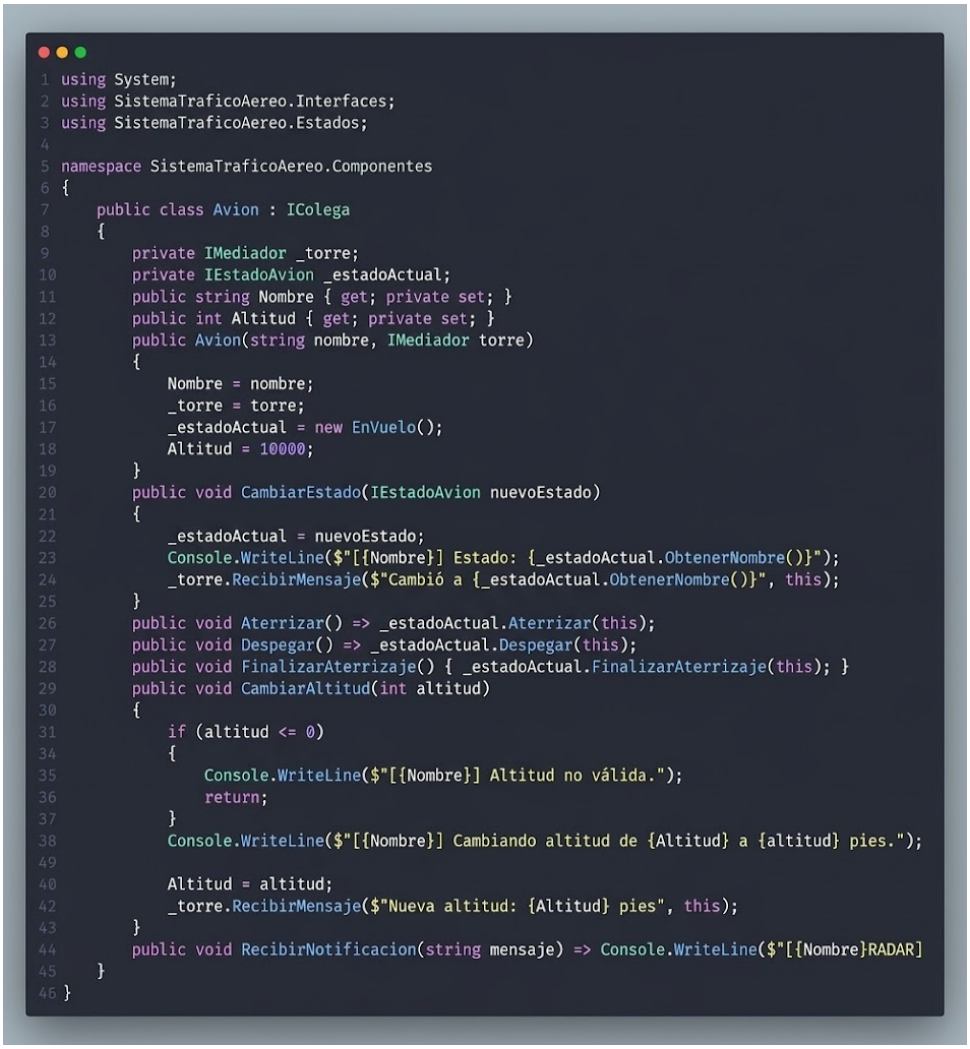
        public void RegistrarAvion(IColega avion)
        {
            lock (_lock) { _aviones.Add(avion); }
            Console.WriteLine($"[TORRE] Registrado: {avion.Nombre}");
        }

        public void EncolarComando(IComando comando) {
            lock (_lock) { _colaComandos.Enqueue(comando); }
            Console.WriteLine($"[TORRE] Encolado: {comando.ObtenerDescripcion()}");
        }

        public void ProcesarComandos()
        {
            lock (_lock) {
                while (_colaComandos.Count > 0) {
                    var cmd = _colaComandos.Dequeue();
                    Console.WriteLine($"\n[TORRE] Ejecutando: {cmd.ObtenerDescripcion()}");
                    cmd.Ejecutar();
            }   }
        }

        public void RecibirMensaje(string mensaje, IColega emisor)
        {
            lock (_lock) {
                Console.WriteLine($"\n[TORRE] De {emisor.Nombre}: {mensaje}");
                foreach (var avion in _aviones) {
                    if (avion != emisor)
                        avion.RecibirNotficacion($"Alerta: {mensaje}");
            }   }
        }
}
```

Figura 22: TorreControl.cs



```
using System;
using SistemaTraficoAereo.Interfaces;
using SistemaTraficoAereo.Estados;

namespace SistemaTraficoAereo.Componentes
{
    public class Avion : IColega
    {
        private IMediador _torre;
        private IEstadoAvion _estadoActual;
        public string Nombre { get; private set; }
        public int Altitud { get; private set; }
        public Avion(string nombre, IMediador torre)
        {
            Nombre = nombre;
            _torre = torre;
            _estadoActual = new EnVuelo();
            Altitud = 10000;
        }
        public void CambiarEstado(IEstadoAvion nuevoEstado)
        {
            _estadoActual = nuevoEstado;
            Console.WriteLine($"[{Nombre}] Estado: {_estadoActual.ObtenerNombre()}");
            _torre.RecibirMensaje($"Cambió a {_estadoActual.ObtenerNombre()}", this);
        }
        public void Aterrizar() => _estadoActual.Aterrizar(this);
        public void Despegar() => _estadoActual.Despegar(this);
        public void FinalizarAterrizaje() { _estadoActual.FinalizarAterrizaje(this); }
        public void CambiarAltitud(int altitud)
        {
            if (altitud <= 0)
            {
                Console.WriteLine($"[{Nombre}] Altitud no válida.");
                return;
            }
            Console.WriteLine($"[{Nombre}] Cambiando altitud de {Altitud} a {altitud} pies.");

            Altitud = altitud;
            _torre.RecibirMensaje($"Nueva altitud: {Altitud} pies", this);
        }
        public void RecibirNotificacion(string mensaje) => Console.WriteLine($"[{Nombre}RADAR]
    }
}
```

Figura 23: Avion.cs

Program



```
using System;
using System.Collections.Generic;
using SistemaTraficoAereo.Componentes;
using SistemaTraficoAereo.Comandos;

namespace SistemaTraficoAereo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== CONTROL DE TRÁFICO AÉREO ===\n");
            TorreControl torre1 = new TorreControl();
            TorreControl torre2 = new TorreControl();
            TorreControl torre3 = new TorreControl();
            List<Avion> flota = new List<Avion>();
            for (int i = 1; i <= 10; i++)
            {
                TorreControl torreAsignada;
                if (i <= 4)
                {
                    torreAsignada = torre1;
                else if (i <= 7)
                  { torreAsignada = torre2; }
                else
                  { torreAsignada = torre3; }

                var avion = new Avion($"Vuelo-{i:D3}", torreAsignada);
                torreAsignada.RegistrarAvion(avion);
                flota.Add(avion);
            }
            // Comandos para la primera torre
            torre1.EncolarComando(new ComandoAterrizar(flota[0]));
            torre1.EncolarComando(new ComandoCambiarAltitud(flota[1], 30000));
            torre1.EncolarComando(new ComandoDespegar(flota[0]));

            // Comandos para la segunda torre
            torre2.EncolarComando(new ComandoDespegar(flota[4]));
            torre2.EncolarComando(new ComandoAterrizar(flota[5]));

            // Comandos para la tercera torre
            torre3.EncolarComando(new ComandoCambiarAltitud(flota[7], 35000));
            torre3.EncolarComando(new ComandoAterrizar(flota[8]));

            // Procesamiento de comandos
            Task tarea1 = Task.Run(() => torre1.ProcesarComandos());
            Task tarea2 = Task.Run(() => torre2.ProcesarComandos());
            Task tarea3 = Task.Run(() => torre3.ProcesarComandos());

            Task.WaitAll(tarea1, tarea2, tarea3);

            Console.WriteLine("\n=== FINALIZACIÓN DE ATERRIZAJES ===");
            flota[0].FinalizarAterrizaje();
            flota[5].FinalizarAterrizaje();
            flota[8].FinalizarAterrizaje();
            Console.ReadKey();
        }
    }
}
```

Figura 24: Program.cs

Salida de Prueba

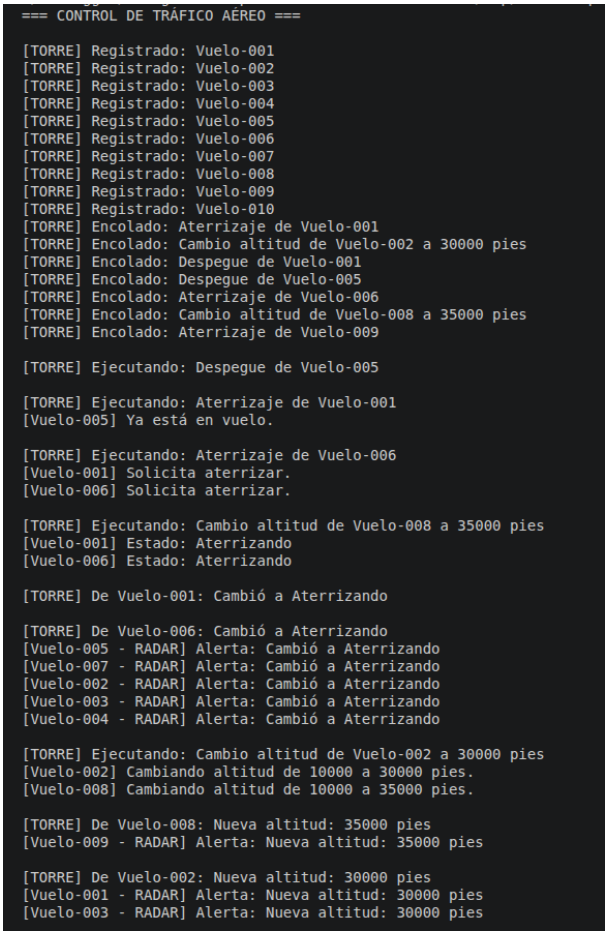


Figura 25: Salida de Prueba parte 1

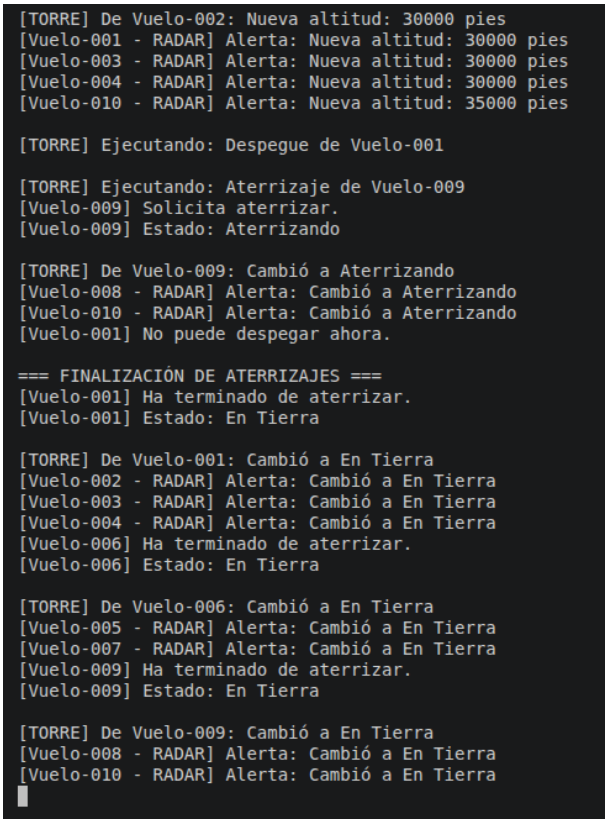


Figura 26: Salida de Prueba parte 2

c) Analiza: ¿Qué pasa si la torre de control falla? ¿Cómo añadir redundancia sin romper el patrón? ¿Es un punto único de fallo?

¿Qué pasa si la torre de control falla? ¿Es un punto único de fallo?

Sí, cada instancia de TorreControl representa un punto único de fallo para el grupo de aviones que tiene asignado, porque es la encargada de coordinar su comunicación y gestionar los comandos que recibe.

En el patrón Mediator, TorreControl centraliza la coordinación de la comunicación, el registro de la flota (_aviones) y el encolamiento de las solicitudes (_colaComandos). Si una torre sufre un fallo crítico, los aviones asignados a ella quedarían sin poder comunicarse a través de su mediador. Además, se interrumpiría el procesamiento de los comandos pendientes en su cola, impidiendo coordinar correctamente las operaciones de aterrizaje, despegue y cambio de altitud. Las otras dos torres podrían continuar funcionando de manera independiente.

¿Cómo añadir redundancia sin romper el patrón?

Una solución sería implementar una torre de control principal y otra de respaldo (Standby Mediator)para cada torre o grupo de aviones.

La torre secundaria mantendría una copia de la información importante de la principal, como los aviones registrados y los comandos pendientes. De esta manera, si la torre principal falla, la secundaria podría asumir sus funciones.

Para conseguirlo, los aviones seguirían comunicándose mediante la interfaz IMediador, sin necesidad de conocer qué torre está trabajando en ese momento. Un mecanismo externo se encargaría de detectar el fallo y cambiar la referencia hacia la torre de respaldo.

Así se mantiene el patrón Mediator, ya que los aviones siguen comunicándose únicamente a través de

una torre de control y no necesitan establecer conexiones directas entre ellos.

d) Integra con Command para encapsular solicitudes y con State para modelar estados del avión (en vuelo, aterrizando, en tierra).
Restricción: El sistema debe manejar mensajes concurrentes y garantizar orden FIFO por avión.

El sistema de tráfico aéreo integra de forma cohesiva tres patrones de diseño fundamentales para resolver problemas de acoplamiento, orden de ejecución y flujos de comportamiento dinámicos:

1. Mediator (TorreControl / IMediador)

La TorreControl se encarga de coordinar la comunicación entre los aviones. Cada avión mantiene una referencia al mediador y utiliza esta interfaz para enviar sus solicitudes y recibir notificaciones. De esta manera, los aviones no necesitan comunicarse directamente entre ellos. En la implementación se utilizan tres instancias de TorreControl, cada una encargada de coordinar un grupo de aviones.

2. Command (IComando y sus implementaciones)

Este patrón permite representar las solicitudes como objetos independientes. Se utilizan comandos como ComandoAterrizar, ComandoDespegar y ComandoCambiarAltitud.

Cuando un avión necesita realizar una operación, se crea el comando correspondiente y se envía a la torre de control correspondiente, que los almacena en una cola para ejecutarlos posteriormente, separando así la solicitud de su ejecución.

3. State (IEstadoAvion y sus estados concretos)

Este patrón permite controlar el comportamiento de cada avión según su estado actual. Se utilizan tres estados: EnVuelo, Aterrizando y EnTierra.

Cada estado determina qué operaciones puede realizar el avión. Por ejemplo, un avión que está en tierra puede despegar, mientras que uno que se encuentra en vuelo puede solicitar aterrizar. Además, el cambio de estado modifica el comportamiento permitido del avión sin necesidad de utilizar grandes estructuras de condicionales.

Manejo de concurrencia y orden FIFO

Como varios aviones pueden enviar solicitudes al mismo tiempo, el sistema utiliza mecanismos de sincronización, como lock, para proteger el acceso a las listas y colas de comandos de cada torre.

Para procesar las solicitudes simultáneamente, se utilizan tareas independientes mediante Task.Run(), permitiendo que las tres torres ejecuten sus respectivas colas al mismo tiempo. Posteriormente, Task.WaitAll() espera a que todas las tareas terminen antes de continuar con la finalización de los aterrizajes.

El orden FIFO se mantiene dentro de cada torre porque los comandos se almacenan en una cola Queue<IComando>y se procesan secuencialmente. Como cada avión está asignado a una sola torre, se conserva el orden de ejecución de sus comandos dentro de esa cola.

Flujo de Colaboración Integrado:

1- El sistema crea los aviones y los registra en sus respectivas torres de control.
2- Se crea el comando correspondiente a cada solicitud, como aterrizar, despegar o cambiar la altitud.
3- La torre de control almacena los comandos recibidos en su cola.
4- Las tres torres procesan sus comandos concurrentemente, respetando el orden FIFO de sus respectivas colas.
5- Al ejecutarse un comando, se invoca la operación correspondiente del avión.

6- El avión consulta su estado actual mediante IEstadoAvion para comprobar si la operación está permitida.

7- Finalmente, se actualiza el estado del avión y la torre notifica el cambio a los demás aviones de su grupo.

Con esta integración, Mediator coordina la comunicación, Command organiza las solicitudes y State controla las operaciones permitidas según la situación de cada avión.